Closures and Higher-order Functions:

1.

```
struct Student {
    let name: String
    let marks: Int
    let dept: String
}

let students = [
    Student(name: "Asha", marks: 82, dept: "IT"),
    Student(name: "Ravi", marks: 35, dept: "CSE"),
    Student(name: "Meena", marks: 91, dept: "IT"),
    Student(name: "Kiran", marks: 56, dept: "ECE"),
    Student(name: "Divya", marks: 67, dept: "CSE"),
    Student(name: "Arjun", marks: 40, dept: "IT")
]
let passedStudents = students.filter { $0.marks >= 40 }
print("Passed Students:")
passedStudents.forEach { print($0.name) }
let names = students.map { $0.name }
print("\nStudent Names:", names)
let totalMarks = students.map { $0.marks }.reduce(0, +)
let average = Double(totalMarks) / Double(students.count)
print("\nAverage Marks:", average)
let rankedStudents = students.sorted { $0.marks > $1.marks }
print("\nStudents Ranked by Marks:")
rankedStudents.forEach {
    print("\($0.name): \($0.marks)")
}
let groupedStudents = Dictionary(grouping: students, by: \.dept)
print("\nStudents Grouped by Department:")
for (department, students) in groupedStudents {
```

```
    print("\(department):", students.map { $0.name })
}
let passed1 = students.filter { student in
    return student.marks >= 40
}
let passed2 = students.filter { $0.marks >= 40 }
```

```
Output
Asha
Meena
Kiran
Divya
Arjun

Student Names: ["Asha", "Ravi", "Meena", "Kiran", "Divya",
"Arjun"]

Average Marks: 61.833333333333336

Students Ranked by Marks:
Meena: 91
Asha: 82
Divya: 67
Kiran: 56
Arjun: 40
Ravi: 35

Students Grouped by Department:
CSE: ["Ravi", "Divya"]
IT: ["Asha", "Meena", "Arjun"]
ECE: ["Kiran"]
```

2.
```
func myFilter(
    _ items: [Int],
    _ predicate: (Int) -> Bool
) -> [Int] {
    var result: [Int] = []

    for item in items {
        if predicate(item) {
            result.append(item)
        }
    }

    return result
}
```

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

let evenNumbers = myFilter(numbers) { number in
    number % 2 == 0
}

print("Even Numbers:", evenNumbers)
func applyTwice(
    _ value: Int,
    _ f: (Int) -> Int
) -> Int {
    return f(f(value))
}

let result = applyTwice(3) { number in
    number * 2
}

print("Apply Twice Result:", result)
class Button {
    private var handlers: [() -> Void] = []

    func onTap(_ handler: @escaping () -> Void) {
        handlers.append(handler)
    }

    func tap() {
        for handler in handlers {
            handler()
        }
    }
```

```
}

let button = Button()

button.onTap {
    print("First handler called")
}

button.onTap {
    print("Second handler called")
}

button.tap()
```

Output

```
Even Numbers: [2, 4, 6, 8, 10]
Apply Twice Result: 12
First handler called
Second handler called
```

```
3. func makeCounter() -> () -> Int {
    var count = 0

    return {
        count += 1
        return count
    }
}

let a = makeCounter()
let b = makeCounter()
```

print(a(), a(), b(), a())

Output

```
1 2 1 3
```